Requisitos: Checa AI

Requisitos funcionais e não funcionais do **Checa AI**, assistente de checagem de fatos em português do Brasil que estima a **propensão** de uma notícia ser desinformação (baixa, média, alta ou indeterminada), com as fontes e o passo a passo da análise.

Versão: 2.0, 14/10/2026. **Código de referência:** `main` @ `d6da038` (pull requests até o nº 21).

Situação no código:

	total	implementado	parcial	não implementado
Requisitos funcionais	**21**	14	7	0
Requisitos não funcionais	**13**	8	5	0
Regras de negócio	**17**	—	—	1 (RN04)

9 pendências dependem de decisão da equipe (seção 5). Equipe: Thiago Correia Gonzaga, Gabriella Oliveira de Souza Dias, Maria Eduarda Denis Duarte Marques, Guilherme Silva Dutra, Eliane Orlandin do Carmo, Daltro Oliveira Vinuto.

Status. *Implementado*: o comportamento existe na `main` e cumpre a descrição. *Parcial*: existe, com a lacuna indicada. *Não implementado*: não existe no código.

Prioridade. *Alta*: sem ela o produto não cumpre o objetivo. *Média*: melhora a qualidade ou a confiança da resposta. *Baixa*: complementar.

Sumário

1. Introdução
2. Requisitos funcionais
3. Requisitos não funcionais
4. Regras de negócio
5. Pendências que precisam de decisão
6. Histórico de alterações em relação à versão anterior (30/09/2026)

---

1. Introdução

Objetivo. O Checa AI ajuda uma pessoa a decidir se compartilha uma notícia que recebeu. Ela envia o texto, o título ou o link, e o sistema responde com a **propensão** de aquele conteúdo ser desinformação (baixa, média, alta ou indeterminada), as fontes que embasam a estimativa e o passo a passo da análise. O sistema nunca emite veredito.

Escopo. Texto em português do Brasil, recebido por um robô no Telegram, por uma página na internet ou pela interface de programação. A evidência vem de checagens publicadas por agências (base própria e Google Fact Check Tools) e de notícias encontradas na web, lidas e julgadas por um modelo de linguagem. Fora do escopo: análise de imagem, vídeo e áudio; outros idiomas; outros aplicativos de mensagem.

Público. Pessoas leigas que recebem notícias em aplicativos de mensagem (público principal), estudantes, professores e jornalistas de checagem.

---

2. Requisitos funcionais

ID	Nome	Descrição	Prioridade	Status	Onde está implementado
RF01	Atender pelo Telegram	Receber mensagens e respondê-las por um robô no Telegram. O comando `/start` apresenta o que o robô faz, as entradas aceitas e o aviso de privacidade; cada consulta recebe confirmação imediata e até 3 atualizações de progresso; só uma instância do robô fica ativa por vez.	Alta	Implementado	`factcheck_mvp/telegram_bot.py`
RF02	Aceitar texto, título ou link	Classificar a entrada em link (começa com `http`/`www`), título (até 140 caracteres em uma linha) ou texto (até 20.000 caracteres). O link só é lido quando o endereço, inclusive depois de redirecionamentos, pertence a um portal do catálogo; quando não pode ser lido, o sistema pede que a pessoa cole o texto.	Alta	Implementado	`telegram_bot.py` (`classificar_entrada`, `_texto_link`), `api.py`
RF03	Recusar mídia com orientação	Responder a imagem, vídeo, arquivo, áudio ou mensagem de voz com uma orientação (enviar o texto; fazer busca reversa da imagem), sem enviar a mídia para análise.	Alta	Implementado	`telegram_bot.py` (`_midia`, `MIDIA_MSG`)
RF04	Consultar checagens existentes	Procurar, para cada afirmação, checagens já publicadas: (a) na base própria de checagens (21.976 registros de 9 agências, com selo tipado); (b) na Google Fact Check Tools, quando houver chave configurada; (c) na web, com consulta dirigida aos domínios das agências. O selo original da agência é preservado.	Alta	Implementado	`indice.py`, `ingestor.py`, `selos.py`, `factcheck_api.py`, `agente.py`
RF05	Consultar veículos de comunicação	Buscar na web (SerpAPI) o que os veículos publicaram sobre cada afirmação e ler o conteúdo das páginas encontradas, não só o título. Sem chave de busca, o sistema segue só com as checagens e declara a limitação.	Alta	Parcial: o catálogo tem 23 veículos gerais, contra a meta de 50 (ver pendência 2)	`serpapi_layer.py`, `agente.py`, `aprofundar.py`, `extracao.py`
RF06	Comparar as fontes	Agrupar republicações e veículos do mesmo grupo como uma única fonte e apontar divergências de selo e de data entre as fontes.	Alta	Implementado	`corroboracao.py`, `decisao.py`

ID	Nome	Descrição	Prioridade	Status	Onde está implementado
RF07	Apresentar as fontes	Mostrar até 3 fontes na resposta (as demais no relatório completo), cada uma com veículo, o que ela faz frente à afirmação (confirma, contesta ou só relata), selo da agência, trecho citado, endereço, se o texto foi lido por inteiro e o nível de confiabilidade do site. Páginas iniciais e de seção não aparecem como evidência.	Alta	Implementado	`telegram_bot.py` (`formatar`), `api.py`, `pipeline.py`, `confiabilidade.py`
RF08	Submeter o texto ao modelo próprio	Submeter o texto enviado ao modelo de aprendizado de máquina da equipe (BERTimbau calibrado) como sinal de estilo, fora da avaliação (RN11). Sem o modelo configurado, usar um substituto identificado como provisório.	Baixa	Parcial: o modelo é carregado por `FAKE_MODEL_PATH`, que não está configurado na operação atual; o piso de 80 palavras (RN04) não está implementado	`modelo_fake.py`
RF09	Exibir a estimativa do modelo	Exibir a probabilidade calibrada do modelo, com o nome do modelo, como "sinal de estilo do texto (não indica veracidade)". Com o substituto provisório, exibir "em treino" sem número.	Baixa	Parcial: depende de RF08	`pipeline.py`, `telegram_bot.py`
RF10	Identificar padrões de desinformação	Apontar, numa lista fechada (apelo à urgência, tom alarmista, fonte vaga, ausência de data ou local, título sensacionalista), os padrões presentes no texto, com o trecho que os evidencia, fora da avaliação (RN11).	Baixa	Implementado: regras fixas por padrão; o modelo de linguagem é opcional (`PADROES_LLM=1`)	`padroes_llm.py`
RF11	Apresentar o passo a passo	Mostrar as etapas executadas com a situação de cada uma (concluída, parcial, pulada, com falha) e listar as limitações da consulta. A falha de uma etapa não interrompe a resposta.	Alta	Implementado	`pipeline.py`, `agregador.py`, `telemetria.py`
RF12	Apresentar propensão, não veredito	Apresentar a avaliação na escala baixa, média, alta ou indeterminada, com a frase "… propensão de ser fake news", sem nunca afirmar que algo "é falso" ou "é verdade" na conclusão do sistema.	Alta	Implementado	`agregador.py` (`verificar_neutralidade`), `decisao.py`
RF13	Fundamentar a avaliação	Justificar a avaliação com as evidências que aumentam e as que reduzem a propensão. Sem nenhuma fonte que confirme ou conteste, nem selo aplicável, a avaliação é "indeterminada".	Alta	Implementado	`decisao.py` (`decidir`), `juiz_llm.py`

ID	Nome	Descrição	Prioridade	Status	Onde está implementado
RF14	Dar elementos para decisão autônoma	Encerrar toda resposta com 3 perguntas-guia (a data é atual? quem assina? aparece em mais de um veículo independente?) e com a frase "Isso não é um veredito".	Média	Parcial: a página traz a frase; no robô, a resposta mostra só o resumo de uma linha, que não a inclui (o aviso aparece só no `/start`)	`agregador.py` (`perguntas_guia`), `decisao.py`, `api.py`
RF15	Oferecer página e interface de programação	Oferecer a mesma consulta por uma página na internet (`GET /`) e por uma interface de programação com entrada e saída tipadas (`POST /checar`), além de `GET /saude` e `GET /portais`.	Alta	Implementado	`api.py`, `schemas.py`
RF16	Registrar a avaliação da pessoa	Oferecer os botões "👍 Ajudou" e "👎 Não ajudou" após a resposta e gravar o voto com o texto consultado e a propensão, sem nome nem identificador da pessoa.	Média	Parcial: só no Telegram; a página na internet não tem os botões	`telegram_bot.py` (`registrar_feedback`)
RF17	Priorizar a base de checagens	Julgar primeiro as checagens da base; pular a busca na web quando houver checagem aplicável (página lida, que confirma ou refuta a mesma afirmação, com citação verificada, selo com direção e data compatível) e registrar "web pulada: checagem aplicável na base".	Alta	Implementado	`pipeline.py` (`_fase_base`), `aplicabilidade.py`
RF18	Informar onde a evidência foi encontrada	Informar se a avaliação veio da base de checagens ou de outras fontes da web, com aviso explícito quando a afirmação não foi encontrada na base.	Média	Parcial: a página e a interface informam; o robô do Telegram não	`schemas.py` (`onde_encontrado`), `api.py`
RF19	Considerar a data da afirmação	Comparar marcadores temporais do texto ("hoje", "ontem", "nesta semana", "agora", "acaba de") com a data de publicação das fontes; fonte de outro período perde peso e recebe o aviso "anterior ao período do texto".	Média	Implementado	`aplicabilidade.py`, `decisao.py`, `telegram_bot.py`
RF20	Sinalizar mensagem não checável como fato	Declarar como limitação, e não classificar como fato: relato de segunda mão ("ouvi dizer"), opinião ou sátira, e afirmação vaga (comparação sem indicador ou período), pedindo à pessoa o dado que falta.	Média	Implementado	`pipeline.py` (`RUMOR_RE`, `OPINIAO_SATIRA_RE`, `VAGO_RE`), `decisao.py`
RF21	Atualizar a base de checagens	Atualizar a base de checagens todo dia, com as publicações novas das agências (RSS, API do WordPress e mapas do site), e submeter a atualização a revisão humana antes de entrar no sistema.	Média	Parcial: o fluxo existe, mas ainda não rodou no GitHub (falta liberar a permissão de criar pedidos de integração)	`.github/workflows/atualizar-base.yml`, `ingestor.py`

3. Requisitos não funcionais

ID	Categoria	Nome	Critério mensurável	Prioridade	Status	Onde está implementado
RNF01	Transparência	Explicar como a avaliação foi obtida	100% das respostas trazem etapas, fontes e limitações; a avaliação é calculada por uma única função determinística com parâmetros documentados, e não por um modelo de linguagem.	Alta	Implementado	`decisao.py` (`decidir`), `schemas.py`; `tests/test_pipeline.py`, `tests/test_decisao.py`
RNF02	Neutralidade	Apresentar a avaliação com imparcialidade	0 ocorrências, no texto produzido pelo sistema, de conclusão binária ("é falso", "é verdade"), adjetivo de julgamento ou termo partidário; selo de agência só aparece atribuído a ela (RN17).	Alta	Implementado	`agregador.py` (`verificar_neutralidade`); `tests/test_schemas.py`
RNF03	Rastreabilidade	Registrar a origem de cada fonte e de cada etapa	Toda fonte exibida tem endereço, veículo e trecho citado; toda execução grava um registro em `runs/<id>/trace.jsonl` com etapas, fontes, chamadas ao modelo de linguagem e motivos de descarte; site novo descoberto vai para `catalogo_propostas.json` e não vale como fonte curada sem aprovação.	Alta	Implementado	`telemetria.py`, `cli.py trace`, `descoberta_site.py`
RNF04	Usabilidade	Redigir em linguagem simples	A 1ª linha da resposta traz a avaliação com símbolo de cor; a 2ª, o motivo em uma frase; nenhum termo interno ("avaliador", "corpo lido", nomes de módulos) aparece para a pessoa; a linguagem é validada com ao menos 5 pessoas do público principal.	Média	Parcial: a resposta ainda mostra termos internos nas etapas ("avaliador", "Novos sites no catálogo"), e não houve validação com o público	`agregador.py`, `telegram_bot.py`
RNF05	Manutenibilidade	Separar a lógica de checagem dos canais	O fluxo de checagem não importa nenhum módulo de canal; robô, página e interface usam as mesmas estruturas de entrada e saída; modelo próprio, índice e modelo de linguagem são trocados por configuração, sem alterar o fluxo.	Alta	Implementado	`pipeline.py`, `schemas.py`, `modelo_fake.py`, `llm.py`

ID	Categoria	Nome	Critério mensurável	Prioridade	Status	Onde está implementado
RNF06	Desempenho	Responder dentro dos limites de tempo e de custo	(a) Mediana do tempo de resposta abaixo de 30 s; (b) até 30 requisições por minuto por endereço de rede na interface; (c) no máximo 100 buscas na web por dia (configurável); (d) buscas repetidas em até 1 hora vêm de cache.	Alta	Parcial: (b), (c) e (d) atendidos; (a) não: em 14/10, 14 s quando a base resolve e 44 a 52 s quando a consulta vai à web	`api.py` (`_rate_ok`), `serpapi_layer.py`, `config.py`
RNF07	Segurança e privacidade	Proteger os dados e o sistema contra uso indevido	Nenhuma chave de serviço no código, no histórico do repositório ou nos registros de execução (redação automática); link enviado pela pessoa só é lido se for de portal do catálogo, com até 1,5 MB e 15 s; julgamento de fonte só vale com trecho citado que exista na página (semelhança ≥ 0,9); aviso de privacidade no início da conversa; o texto das páginas é delimitado no prompt e tratado como dado.	Alta	Parcial: não há delimitação explícita contra injeção de instruções no prompt, nem prazo de retenção dos registros (`runs/`, `feedback.jsonl`)	`telemetria.py` (`redigir`), `telegram_bot.py`, `juiz_llm.py`, `config.py`
RNF08	Confiabilidade	Manter a resposta diante de falhas	A falha de qualquer etapa vira limitação declarada, sem interromper a resposta; a busca na web tem 1 nova tentativa em queda de conexão; o modelo de linguagem tem modelo reserva e, por último, regras fixas; sem julgamento das fontes, a resposta é "indeterminada" (RN12).	Alta	Implementado	`pipeline.py`, `serpapi_layer.py`, `llm.py`, `llm_openrouter.py`
RNF09	Ética	Medir e mitigar vieses do modelo e dos dados	Toda avaliação do modelo relata acerto por grupo de origem, por canal e por tamanho do texto; metas da próxima versão: acurácia no pior grupo ≥ 0,70, acerto em notícias verdadeiras com menos de 80 palavras ≥ 0,85, erro de calibração ≤ 0,05.	Média	Parcial: medido na versão R0 (pior grupo 0,575); metas não atingidas; viés por tema e posição política não medido	Repositório `thiago-cg/FakenewsBR`; `eval/`
RNF10	Compatibilidade	Respeitar os limites do canal	Toda mensagem do robô tem no máximo 4.096 unidades UTF-16 (limite do Telegram) e nunca é cortada no meio de um endereço.	Alta	Implementado	`telegram_bot.py` (`_cortar_telegram`)
RNF11	Manutenibilidade	Verificar cada mudança automaticamente	100% dos testes automatizados passam antes de cada integração (hoje 1.214); os testes rodam sozinhos a cada envio ao repositório; a avaliação do sistema é reproduzível sem rede, por gravação das chamadas externas.	Média	Parcial: os testes rodam à mão; não há integração contínua para os testes	`tests/`, `eval/`, `replay.py`

ID	Categoria	Nome	Critério mensurável	Prioridade	Status	Onde está implementado
RNF12	Operabilidade	Expor o estado do serviço	`GET /saude` informa o uso diário da busca e o teto, e a validade da chave do modelo de linguagem, com aviso a menos de 5 dias do vencimento; o robô avisa quando a chave é recusada; toda variável de configuração lida pelo código está documentada no exemplo de configuração (verificado por teste).	Média	Implementado	`api.py`, `llm_openrouter.py` (`status_chave`), `.env.mvp.example`
RNF13	Qualidade dos dados	Manter a qualidade da base de checagens	No conjunto de referência do índice, a checagem correta aparece entre as 3 primeiras em ≥ 75% das consultas e a taxa de falso positivo é ≤ 10%; a atualização diária é barrada quando o critério falha; checagens reservadas para avaliação nunca entram na base.	Alta	Implementado: 0,86 e 9,2% em 14/10	`scripts/bench_indice_checagens.py --gate`, `checagens_holdout_excluidas.json`

---

4. Regras de negócio

ID	Regra	Requisitos
RN01	A resposta nunca é binária: só a escala de propensão.	RF12, RNF02
RN02	A posição da fonte frente à afirmação prevalece sobre o selo: um selo "verdadeiro" numa página que refuta a afirmação conta contra ela.	RF13
RN03	Republicações do mesmo texto, aliases do mesmo portal e veículos do mesmo grupo econômico contam como uma única fonte.	RF06
RN04	O modelo próprio só opina em textos com 80 palavras ou mais; abaixo disso, a etapa é registrada como pulada, com o motivo. **Não implementado** (ver pendência 3).	RF08
RN05	Páginas iniciais, de seção e de listagem (paginação, tag, autor) não são evidência.	RF07
RN06	Nenhum domínio é bloqueado por lista: o filtro é a relevância para a afirmação. Redes sociais e PDFs não são baixados (não há leitura possível), mas aparecem como fonte "só manchete".	RF05, RNF02
RN07	O sistema não contorna acesso pago. **Em conflito com o código** quanto à proteção contra robôs (ver pendência 1).	RF05
RN08	O modelo de linguagem não redige a conclusão: ela vem de modelos de texto fixos.	RF12, RNF01
RN09	Opinião e sátira não são classificadas como fato.	RF20
RN11	O modelo próprio e os padrões textuais não entram na avaliação: aparecem como sinais de estilo.	RF08, RF09, RF10
RN12	Sem julgamento do conteúdo das fontes, a avaliação é "indeterminada".	RF13, RNF08

ID	Regra	Requisitos
RN13	Fonte considerada só pelo título (página não lida) não vota, nem pela posição nem pelo selo; aparece como "só manchete — não analisada integralmente". Selo que vem só do título de um resultado de agência é exibido como "selo da agência no título", sem votar.	RF07, RF13
RN14	Selo de agência só vale quando o juiz indica que a página confirma ou refuta a mesma afirmação; selo de página que só relata, ou de outra alegação, não vota.	RF04, RF13, RF17
RN15	Agência de checagem curada é confiável por natureza: quando ela refuta a afirmação, a refutação pesa como um selo, e a confirmação de fonte que não é agência deixa de votar nessa afirmação.	RF13
RN16	Data incompatível com o marcador temporal do texto reduz o peso da fonte sem zerá-lo, nunca eleva a propensão e impede que a checagem seja considerada aplicável.	RF17, RF19
RN17	Selo de agência é sempre exibido atribuído a ela ("Selo da agência: FALSO"); a conclusão do sistema continua sendo a propensão.	RF07, RNF02
RN18	Fonte de rede social ou de site pouco acessado, sozinha, não leva a avaliação a "alta" nem a "baixa".	RF07, RF13

---

5. Pendências que precisam de decisão

#	Questão	Sugestão
1	**RN07 × código.** A regra diz que o sistema não contorna proteção contra robôs, mas o código usa `curl_cffi` imitando um navegador e lê páginas da AFP e da UOL que bloqueiam acesso automatizado; o `robots.txt` do Aos Fatos bloqueia robôs de inteligência artificial pelo nome.	Reescrever RN07 como "não contorna acesso pago nem pedido de login; lê só páginas públicas, identificando-se como navegador, a no máximo 1 requisição por segundo por site", e retirar da coleta automática os sites cujo `robots.txt` bloqueie o agente usado.
2	**Meta de 50 veículos (RF05).** Com a busca aberta na web, o catálogo deixou de limitar quais veículos são consultados; hoje ele define só a confiabilidade.	Trocar a meta por "≥ 80% das respostas com ao menos uma fonte de veículo ou agência verificada pela equipe" e marcar RF05 como Implementado.
3	**Piso de 80 palavras (RN04).** Documentado e com variável de configuração, mas não implementado; o modelo não entra na avaliação (RN11).	Implementar (mudança pequena em `modelo_fake.py`) para a resposta não exibir número em texto curto, onde o modelo erra mais.
4	**Metas de desempenho (RNF06).** A mediana de 30 s não é atingida quando a consulta vai à web, e o limite diário de chamadas ao modelo de linguagem (`LLM_DAILY_CAP`) não é lido pelo código. O cache da resposta inteira (`RESULT_CACHE_TTL`) também não existe.	Aceitar 60 s como meta quando há busca na web (30 s quando a base resolve); implementar o limite diário de chamadas e o cache de resposta, ou retirar as duas variáveis da documentação.
5	**Retenção de dados (RNF07).** Não há prazo para apagar `runs/` (texto consultado e chamadas) e `feedback.jsonl`.	Apagar automaticamente após 90 dias e informar o prazo no aviso de privacidade.
6	**Modelo próprio em operação (RF08, RF09).** O código integra o BERTimbau, mas a operação roda o substituto provisório.	Publicar o modelo R0 na operação ou declarar RF08 e RF09 como "fora da versão de entrega" e retirar a linha "Modelo de detecção" da resposta.
7	**"Sem evidência" com RN15.** Com agência curada pesando como selo, um boato sem prova que uma agência refutou sobe para "alta" (decisão 1 do pull request nº 19).	Manter: a refutação da agência é a evidência mais forte que o sistema tem; registrar a limitação na resposta.

#	Questão	Sugestão
8	**Corpo da checagem quando a página não carrega (RN13).** Usar o trecho guardado na base como corpo enfraquece a regra "título não vota" para a base curada (decisão 2 do pull request nº 19).	Permitir só para registros da base com ClaimReview (selo estruturado pela própria agência).
9	**Outro aplicativo de mensagem.** O WhatsApp aparecia como critério de RNF05 e como história de usuário.	Declarar fora do escopo desta entrega (já está na Introdução); RNF05 garante que um novo canal seja só um adaptador.

---

6. Histórico de alterações em relação à versão anterior (30/09/2026)

Status atualizados com o código

Item	Antes	Agora	Motivo
RF02, RF03	Parcial / Atendido com ressalva †	Implementado	Pedido do texto quando o link não abre e orientação para áudio e voz entraram na `main`.
RF04	Parcial	Implementado	Base com 21.976 checagens de 9 agências (antes 8 agências por RSS), mapas do site, atualização diária e Google Fact Check Tools.
RF05	Parcial	Parcial	A lacuna mudou: a busca é aberta; o que falta é a decisão da pendência 2.
RF08	Parcial (piso de 80 palavras na branch de melhorias)	Parcial	O piso não foi integrado (pendência 3).
RF14	Atendido	Parcial	A frase "não é um veredito" saiu da resposta do robô quando ela passou a mostrar só o resumo de uma linha.
RF10	Atendido com modelo de linguagem	Implementado com regras fixas	O modelo de linguagem foi desligado por padrão (marcava a moldura "me mandaram" como fonte vaga).
RNF03	Parcial	Implementado	Site descoberto vai para `catalogo_propostas.json` e não entra no catálogo sem aprovação (P05 resolvida).
RNF06	Limite de 50 chamadas ao modelo de linguagem por dia	Não implementado	`LLM_DAILY_CAP` existe só na documentação (pendência 4).
RNF07	Blindagem contra injeção na branch de melhorias †	Parcial	A blindagem não foi integrada.

Requisitos novos (vindos das decisões de produto de 07/10 a 14/10): RF16 (avaliação da pessoa, antes só IND09/HU13), RF17 (base primeiro, E1), RF18 (onde foi encontrado, E2), RF19 (data da afirmação, E4), RF20 (mensagem não checável), RF21 (atualização diária da base, F1); RNF10 (limite do canal, antes "Restrição"), RNF11 (verificação automática, antes P12), RNF12 (estado do serviço, F5 e F6), RNF13 (qualidade da base); RN13 a RN18 (E3, B1/P2, B4, E4, E5 e confiabilidade das fontes).

Fundidos ou removidos

Item anterior	Destino
RN10 (fonte lida pesa mais que a lida só pelo título)	Fundida em RN13: a fonte lida só pelo título deixou de votar.
RNF06 a RNF09 no documento de produto de inteligência artificial (`versao-anterior/ENGENHARIA_DE_PRODUTO_DE_IA.md`)	Trazidos para a tabela única da seção 3, com critério mensurável.
Critério "um novo canal, como o WhatsApp" (RNF05) e HU15	Fora do escopo (Introdução; pendência 9).
Seção "Restrições"	Viraram RNF10 (limite do canal) e itens de RNF06 e RNF07; "só português" está no escopo.
Histórias de usuário, indicadores, métricas do modelo, matriz de rastreabilidade e lista P01–P19	Não repetidos aqui: continuam no documento anterior; as P que ainda dependem de decisão estão na seção 5.

Termos. "Atendido / Parcial / Pendente" passaram a "Implementado / Parcial / Não implementado". Toda linha ganhou prioridade (Alta, Média, Baixa), que é uma proposta desta versão.